

2025 年度 科学技術計算

学生番号 36714143
情報工学科知能情報分野
和田小百合

2025 年 12 月 26 日

目次

1	課題 06-1	2
1.1	課題の内容	2
1.2	自作関数について	2
1.3	実行結果について	2
2	課題 06-2	3
2.1	課題の内容	3
2.2	それぞれの関数について	3
2.3	実行結果について	5

1 課題 06-1

1.1 課題の内容

最適化手法をテストするための目的関数 $f(x)$ を自分で設定した。自作関数は `jax` を用いた自動微分を利用できるように、`FunctionJax` を継承して実装しました。また、可視化関数 `visualize_f()` を用いて設定した目的関数を描画しました。

1.2 自作関数について

自作した関数は以下のようになります。

```
1
2 #目的関数を作成
3 class AnotherNonConvexFunctionJax(FunctionJax):
4     def forward(self, x: jnp.ndarray) -> float:
5         x0, x1 = x[0], x[1]
6         return (
7             0.05 * (x0**4 + x1**4)
8             - (x0**2 + x1**2)
9             + 0.5 * (x0 + x1)
10        )
```

$f(x, y) = 0.05(x^4 + y^4) - (x^2 + y^2) + 0.5(x + y)$ の関数を作成しました。この関数は 4 次項 ($0.05(x^4 + y^4)$) は $|x|, |y|$ が大きくなると必ず増加し、関数が無限に下がらない構造となっており、2 次項 ($-(x^2 + y^2)$) は原点近くで下にへこんでいます。そして、線形項 ($0.5(x + y)$) は、関数全体を斜め方向に傾け対称性を壊して曲焦点をずらす構造となっていて、最適解が原点からずれます。また、自作関数については自動で微分するために `class FunctionJax()` を継承しています。

1.3 実行結果について

実行したプログラムは以下のようになります。

```
1
2 f = AnotherNonConvexFunctionJax()
3 fig, ax = plt.subplots()
4 visualize_f(f, ax)
5 plt.show()
6
7 f.grad(jnp.array([1.0, 2.0]))
8 f.hessian(jnp.array([1.0, 2.0]))
9 x = jnp.array([1.0, 2.0])
10 print(f.grad(x))
11 print(f.hessian(x))
```

この関数を可視化関数 `visualize_f()` を用いて作成したグラフは以下のようになります。

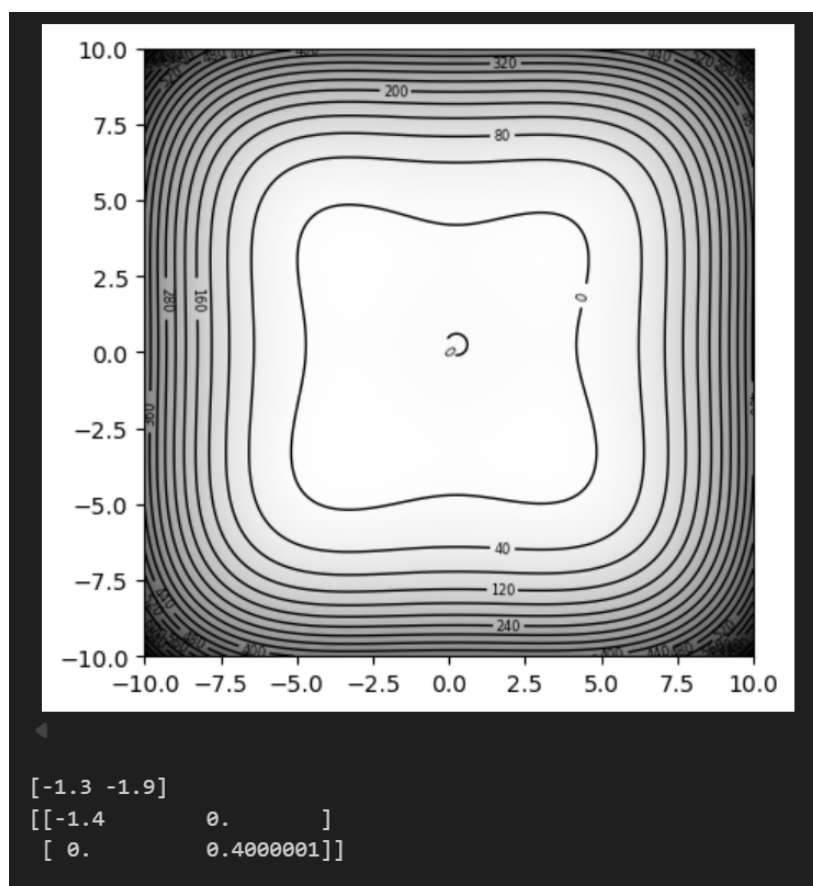


図1 $f(x, y) = 0.05(x^4 + y^4) - (x^2 + y^2) + 0.5(x + y)$ のグラフ

また、 $f(x, y) = 0.05(x^4 + y^4) - (x^2 + y^2) + 0.5(x + y)$ を手計算すると、

$$\nabla f(x, y) = \begin{pmatrix} 0.2x^3 - 2x + 0.5 \\ 0.2y^3 - 2y + 0.5 \end{pmatrix}$$

$$H = \begin{pmatrix} 0.6x^2 - 2 & 0 \\ 0 & 0.6y^2 - 2 \end{pmatrix}$$

となり、実際のプログラムコードと一致することがわかりました。

2 課題 06-2

2.1 課題の内容

準ニュートン法の更新式について収束の速さ奇跡を比較する課題です。

2.2 それぞれの関数について

ヘッセ行列 B を近似して更新する場合 $B^{(k)}\mathbf{d}^{(k)} = -\nabla f(\mathbf{x}^{(k)})$ を解いて $\mathbf{d}^{(k)}$ を求める (つまり $\mathbf{d}^{(k)} = -(B^{(k)})^{-1}\nabla f(\mathbf{x}^{(k)})$) ヘッセ行列の逆行列 H を近似して更新する場合- $\mathbf{d}^{(k)} = -H^{(k)}\nabla f(\mathbf{x}^{(k)})$

それぞれに対して BFGS, DFP, SR1 の更新方法があり、全部で 6 種類の関数を実装しました。

以下が関数のプログラミングコードです。

```

1 def update_matrix(M, s, y, update, method):
2     if method == "BFGS":
3         if update == "H":
4             ys = y @ s

```

```

5         if ys <= 1e-10:
6             return M
7         I = np.eye(len(s))
8         return (
9             (I - np.outer(s, y) / ys)
10            @ M
11            @ (I - np.outer(y, s) / ys)
12            + np.outer(s, s) / ys
13        )
14    else: # B
15        Bs = M @ s
16        ys = y @ s
17        if ys <= 1e-10:
18            return M
19        return (
20            M
21            - np.outer(Bs, Bs) / (s @ Bs)
22            + np.outer(y, y) / ys
23        )
24
25    elif method == "DFP":
26        # の双対BFGS
27        return update_matrix(M, y, s, update, "BFGS")
28
29    elif method == "SR1":
30        if update == "H":
31            v = s - M @ y
32            denom = v @ y
33        else:
34            v = y - M @ s
35            denom = v @ s
36
37        if abs(denom) < 1e-10:
38            return M
39        return M + np.outer(v, v) / denom
40
41    def quasi_newton(
42        f: callable,
43        g: callable,
44        x0: np.ndarray,
45        update: str, # "B" or "H"
46        method: str, # "BFGS", "DFP", "SR1"
47        maxiter: int = 100,
48        callback: callable = None,
49    ) -> np.ndarray:
50
51        x = x0.copy()
52        n = len(x)
53        M = np.eye(n) # B or H
54
55        gk = g(x)
56
57        for k in range(maxiter):

```

```

58
59     # 探索方向
60     if update == "H":
61         dk = -M @ gk
62     else:
63         dk = -np.linalg.solve(M, gk)
64
65     # 直線探索
66     alpha, *_ = scipy.optimize.line_search(f, g, x, dk)
67     if alpha is None:
68         alpha = 1e-3
69
70     x_new = x + alpha * dk
71     g_new = g(x_new)
72
73     s = x_new - x
74     y = g_new - gk
75
76     # 更新
77     M = update_matrix(M, s, y, update, method)
78
79     if callback:
80         callback(x_new)
81
82     # 終了判定
83     if np.linalg.norm(s) < 1e-8 or np.linalg.norm(g_new) < 1e-8:
84         return x_new
85
86     x, gk = x_new, g_new
87
88     return x

```

2.3 実行結果について

結果を比較するために作成したコードは以下のようになります。

```

1  methods = ["BFGS", "DFP", "SR1"]
2  updates = ["B", "H"]
3
4  x0_list = [
5      np.array([-7.0, 7.0]),
6      np.array([-4.0, 2.0]),
7      np.array([0.0, 7.5]),
8  ]
9
10 for f in [
11     AnotherNonConvexFunctionJax(),
12     NonConvexFunction(),
13 ]:
14     callback_list = []
15
16     for x0 in x0_list:
17         for update in updates:

```

```

18     for method in methods:
19
20         label = f"{update}-{method} from {x0}"
21         cache = CacheXk(f, f.grad, x0, label)
22
23         x = quasi_newton(
24             f,
25             f.grad,
26             x0,
27             update=update,
28             method=method,
29             maxiter=50,
30             callback=cache,
31         )
32
33         print(label, "→", x)
34         callback_list.append(cache)
35
36     PlotCacheXk(callback_list).plot_2d()

```

実行結果は以下のようになりました。

図より、FGS 法は B 更新・H 更新のいずれにおいても、目的関数値および勾配ノルムの減少が最も速く、安定した収束を示していることが分かる。一方、DFP 法は BFGS に比べて収束がやや遅く、SR1 法では非凸関数の影響により収束過程が不安定になる場合が確認された。等高線図上の軌跡からも、BFGS 法は谷構造に沿って効率的に移動しているのに対し、SR1 法は曲率近似の不安定性により遠回りする傾向が見られる。

